

Web App. Dev. using PHP

ITE-5330-0GA

Legislation Process Management System for the Federal Parliament of Canada

Assignment 1

28-10-2024

By:

Chelsi Dodia (N01611038)

Jenish Tarsariya (N01604495)

Gurusharan Singh (N01607756)

Contents

Group Details.....	4
Member Info.....	4
Work Distribution.....	4
Legislation Process Management System.....	5
System Design.....	5
Objective.....	5
Project Structure	6
Directories Explanation	7
Key Classes and Methods	10
Class Diagram Overview	10
Class Diagram.....	11
Sequence Diagram.....	12
Data Storage Approach.....	13
Using JSON	13
Storage Location.....	13
Screenshots.....	14
Login Page.....	14
Registration Page.....	14
Incorrect Username/Password.....	15
Successful Login	15
Successful Registration.....	16
User already exists	16
Dashboard (After successful Login)	17
Notifications.....	18
Manage Bills	18
Add Bill.....	19
Demo Bill Creation Notification	20
Vote.....	20
Vote Now	21
Voting as In Favour (Vote Recorder).....	21
Voting Notified.....	22
Set Up and Run the System.....	23

Setup Steps 23

Group Details

Member Info

Name	ID
Chelsi Dodia	N01611038
Jenish tarsariya	N01604495
Gurusharan Singh	N01607756

Work Distribution

Chelsi Dodia	Tech Lead, Full stack, Data Architect, Documentation
Jenish Tarsariya	Full Stack (Frontend, Backend), Data Architect
Gurusharan Singh	Tester, Diagrams

Legislation Process Management System

System Design

Objective

The Legislation Process Management System is designed to streamline and digitize the processes of drafting, reviewing, and voting on legislation within the Federal Parliament of Canada. The system aims to support legislators, administrative staff, and stakeholders by providing an integrated platform for managing legislative documents, facilitating secure voting, and ensuring transparent access to legislative records. The system enhances collaboration and accountability in the legislative process, with the following key objectives:

- **Secure User Authentication and Access Control:**
 - Ensure that only verified users can access, create, or modify legislative documents.
 - Implement role-based access for different user types (e.g., legislators, staff, public) to maintain data security and integrity.
- **Facilitate Transparent Review and Approval Processes:**
 - Support an intuitive interface for managing the lifecycle of a bill, from initial draft to final approval.
 - Allow for changes, comments, and approval requests to flow seamlessly through the system, with clear tracking and notification features for reviewers.
- **Enable Digital Voting and Record-Keeping:**
 - Provide a secure mechanism for voting on legislative proposals, enabling legislators to cast votes on bills electronically.
 - Record and store votes securely to ensure transparency, prevent tampering, and maintain historical records for future reference.
- **Reduce Administrative Burden and Paper Dependency:**
 - Minimize manual paperwork and administrative overhead by digitizing bill submission, review, and voting processes.
 - Store all relevant data electronically, reducing reliance on physical records and supporting eco-friendly practices.

Project Structure

1. Controllers:

- **AuthController:** Manages user authentication, including login and registration functions.
- **BillController:** Handles actions related to bill management, such as creating, editing, deleting, and retrieving bills.
- **VotingController:** Manages voting on bills, including submitting votes and retrieving vote data.

2. Models:

- **Bill:** Manages bill data stored in bills.json, providing methods for CRUD operations.
- **User:** Handles user data stored in users.json, offering functions for user retrieval and registration.
- **Vote:** Manages vote data in votes.json, allowing votes to be stored and retrieved.

3. Data Storage:

- JSON files (bills.json, users.json, and votes.json) store data related to bills, users, and votes, respectively.

Directories Explanation

1. index.php

- **Purpose:** Acts as the entry point for the application, routing requests to the appropriate controller based on user actions.
- **Functionality:**
 - Initializes the environment, loading any necessary dependencies (e.g., from composer.json if using Composer).
 - Loads routes from the routes/web.php file, defining which controller and method to call for each route.
 - Starts the session and handles basic setup for user interactions.

2. routes/web.php

- **Purpose:** Defines application routes, linking specific URLs to controller methods.
- **Structure:**
 - **Authentication Routes:** Define routes for login (AuthController@login) and registration (AuthController@register).
 - **Bill Management Routes:** Direct requests to BillController methods for actions like creating, updating, and viewing bills.
 - **Voting Routes:** Route requests to VotingController for voting actions and retrieving voting results.

3. helpers/session_helper.php

- **Purpose:** Manages session-related functions, providing utilities for handling user login states and permissions.
- **Core Functions:**
 - **checkSession():** Verifies if a user session is active; if not, it redirects to the login page.
 - **logout():** Destroys the current session, effectively logging the user out.
 - **setSessionData():** Sets session variables for logged-in users, such as username or user role.

4. controllers/ (Directory)

- **Purpose:** Handles the core business logic, processing requests and preparing data for the views.
- **Core Controllers:**
 - **AuthController.php:** Manages login and registration, interacting with User model for user authentication and data handling.
 - **BillController.php:** Manages CRUD operations for bills, including retrieving bill data for listing in the views.
 - **VotingController.php:** Manages the voting process, interacting with Vote model to record and retrieve vote data.

5. models/ (Directory)

- **Purpose:** Represents the data structure, encapsulating data handling and interactions with the JSON files in the data/ directory.
- **Core Models:**
 - **User.php:** Handles user data (stored in users.json), including adding and retrieving user information, verifying passwords.
 - **Bill.php:** Manages legislative bill data, handling CRUD operations and storing bills in bills.json.
 - **Vote.php:** Manages vote data, recording user votes and storing them in votes.json.

6. views/ (Directory)

- **Purpose:** Contains the user interface templates, displaying information based on data from controllers.
- **Core Views:**
 - **login.php** and **registration.php:** Form views for user login and registration.
 - **dashboard.php:** Main dashboard view, showing a list of active bills and their statuses.
 - **bill.php** and **billReview.php:** Views for creating, editing, and reviewing bills.
 - **voting.php:** Displays voting options for users to vote on bills.

7. data/ (Directory)

- **Purpose:** Stores application data in JSON format, providing a lightweight alternative to a database.
- **Core Files:**
 - **bills.json:** Stores information on each legislative bill.
 - **users.json:** Holds registered user data, including usernames and hashed passwords.
 - **votes.json:** Records votes on bills, associating each vote with a bill ID and user ID.

8. README.md

- **Purpose:** Provides an overview of the project, including setup instructions, project objectives, and general information about how the system works.

Key Classes and Methods

Class Diagram Overview

Bill

Attributes: id, title, description, status, sponsor, dateIntroduced

Methods:

- createBill(title, description, sponsor)
- updateBill(id, title, description, status)
- getBillDetails(id)

Amendment

Attributes: id, billId, text, dateProposed

Methods:

- proposeAmendment(billId, text)
- getAmendmentDetails(id)

Vote

Attributes: id, billId, voterId, voteType, date

Methods:

- castVote(billId, voterId, voteType)
- getVoteResults(billId)

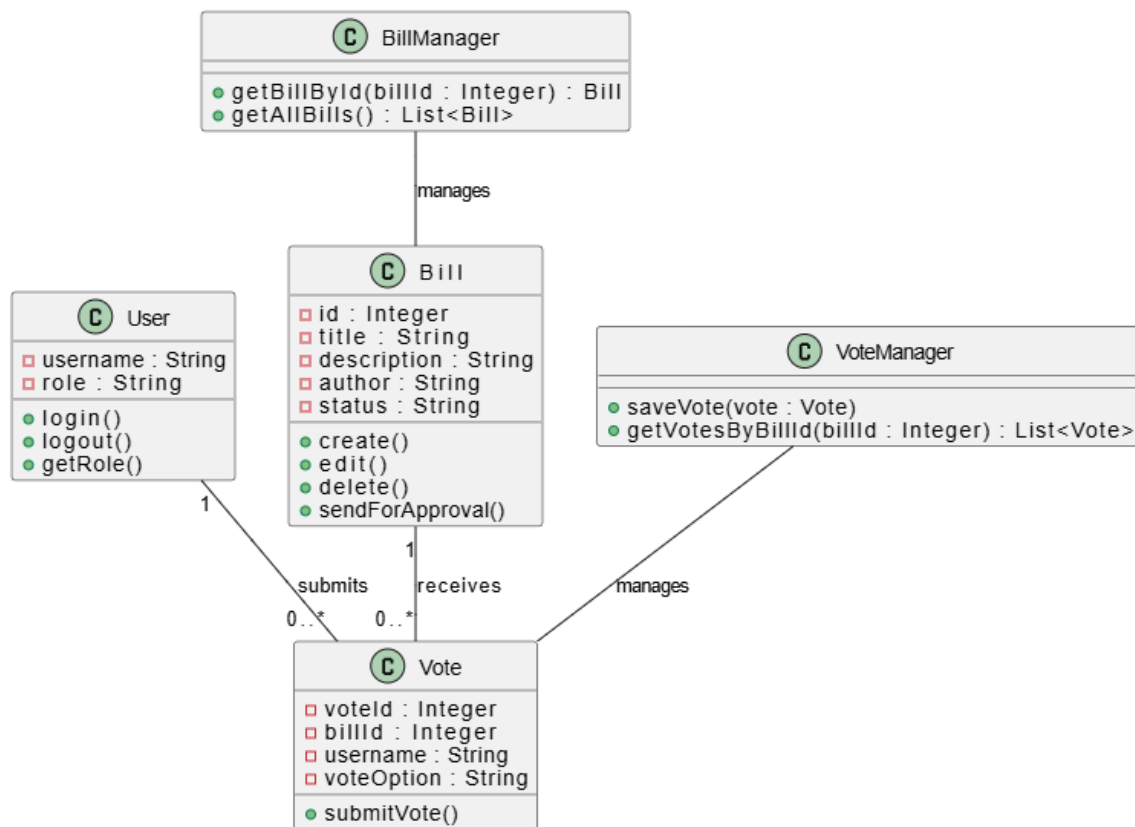
User

Attributes: id, name, role

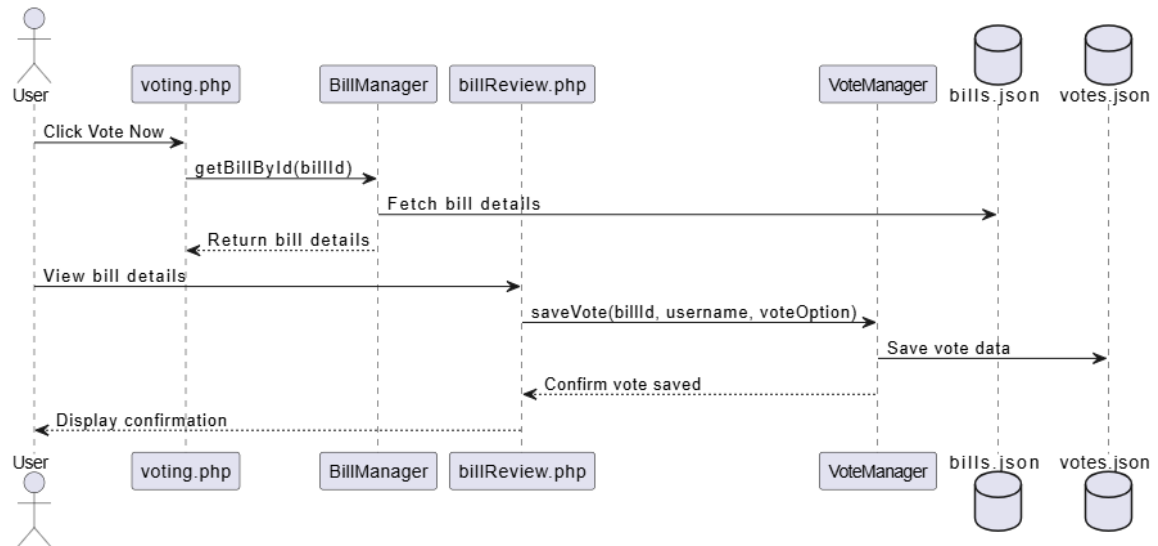
Methods:

- registerUser(name, role)
- getUserDetails(id)

Class Diagram



Sequence Diagram



Data Storage Approach

Using JSON

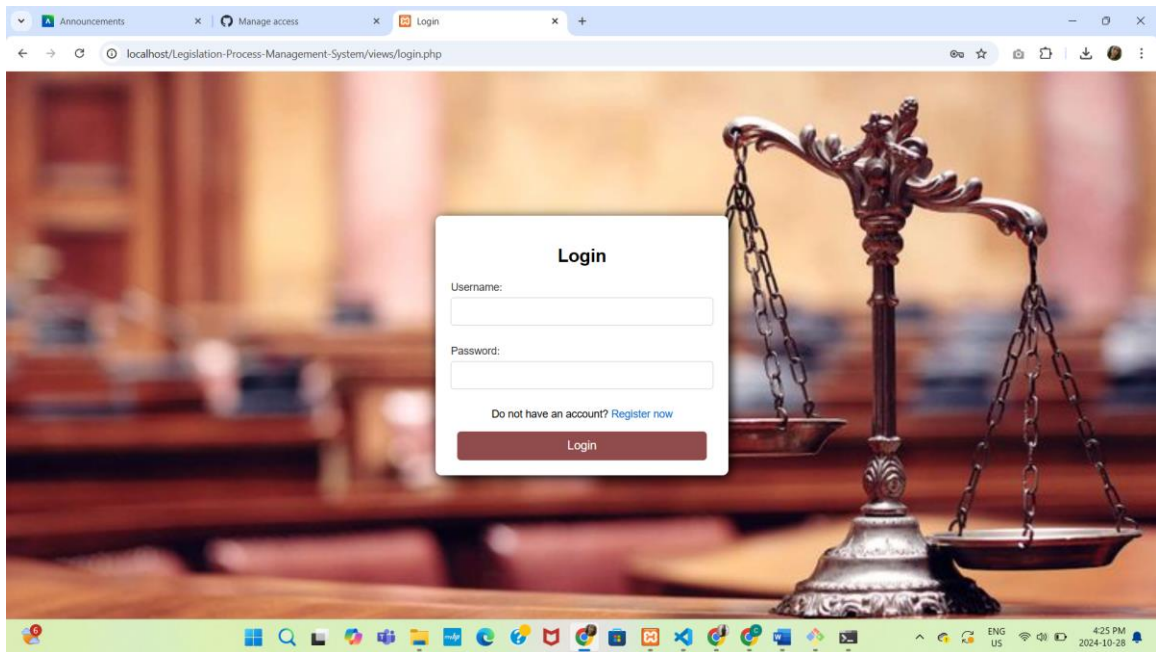
- **Lightweight and Human-Readable:** JSON files are easily editable and readable, allowing for quick debugging and data verification.
- **Portability:** JSON files can be easily transferred and backed up across different environments.
- **Simplicity:** For a system that doesn't require the complexity of relational database management (like MySQL or PostgreSQL), JSON is an ideal choice, especially for development and smaller datasets.

Storage Location

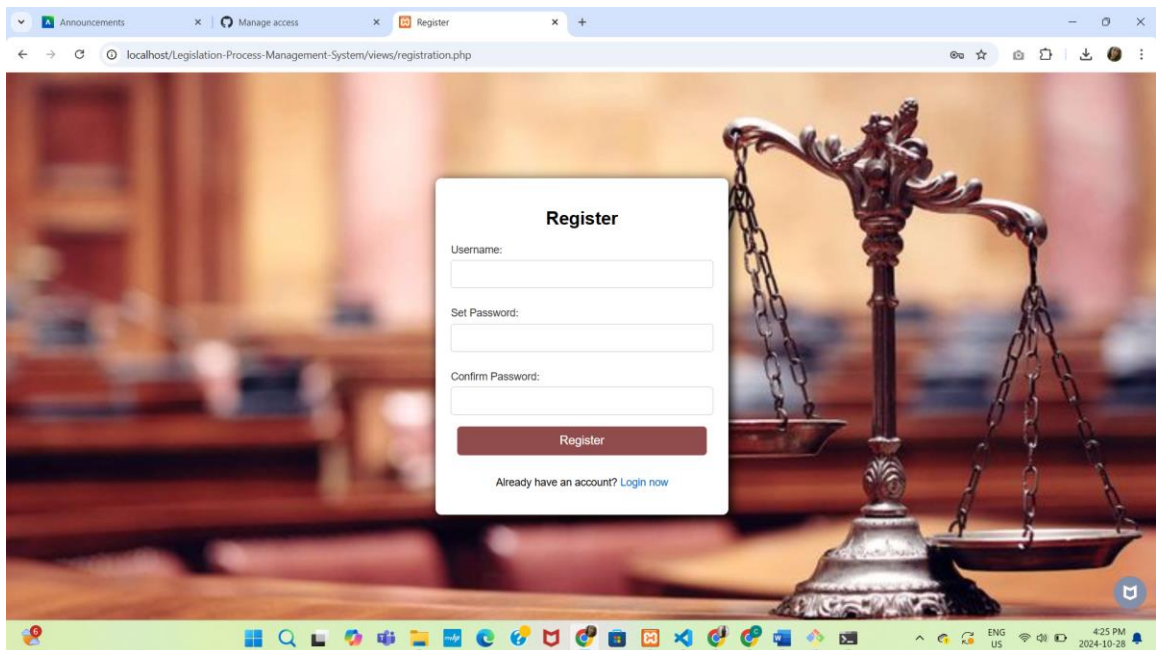
- JSON files are stored in the /data directory.
- Each JSON file represents a distinct data model (e.g., users.json, bills.json, votes.json), helping maintain clear separation of data.

Screenshots

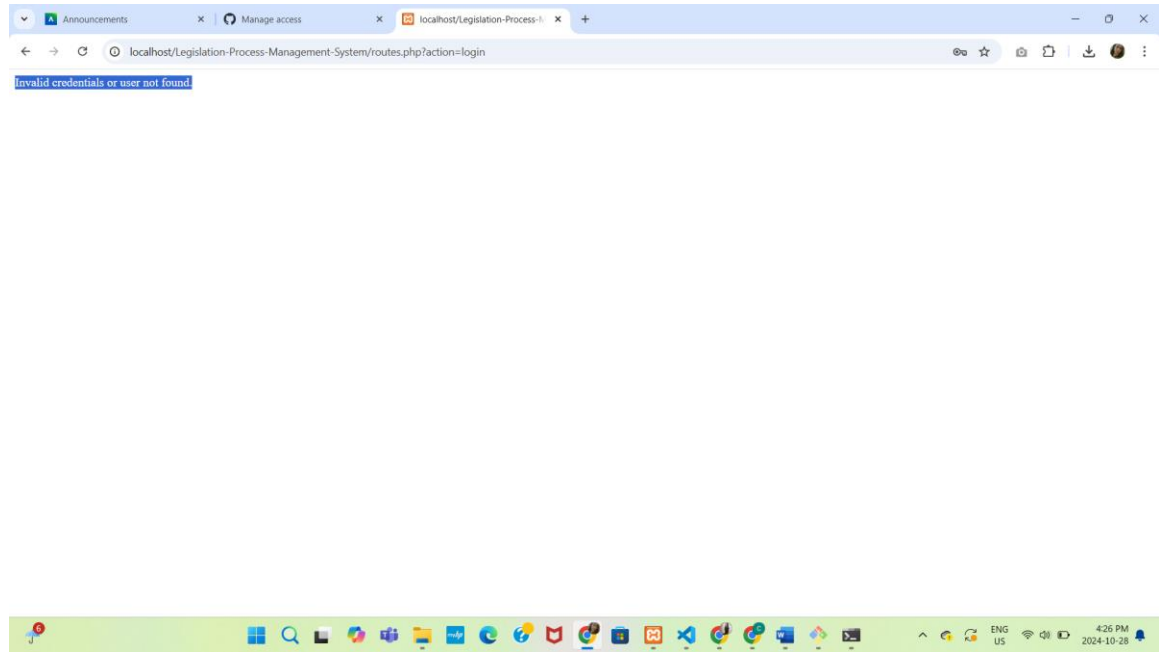
Login Page



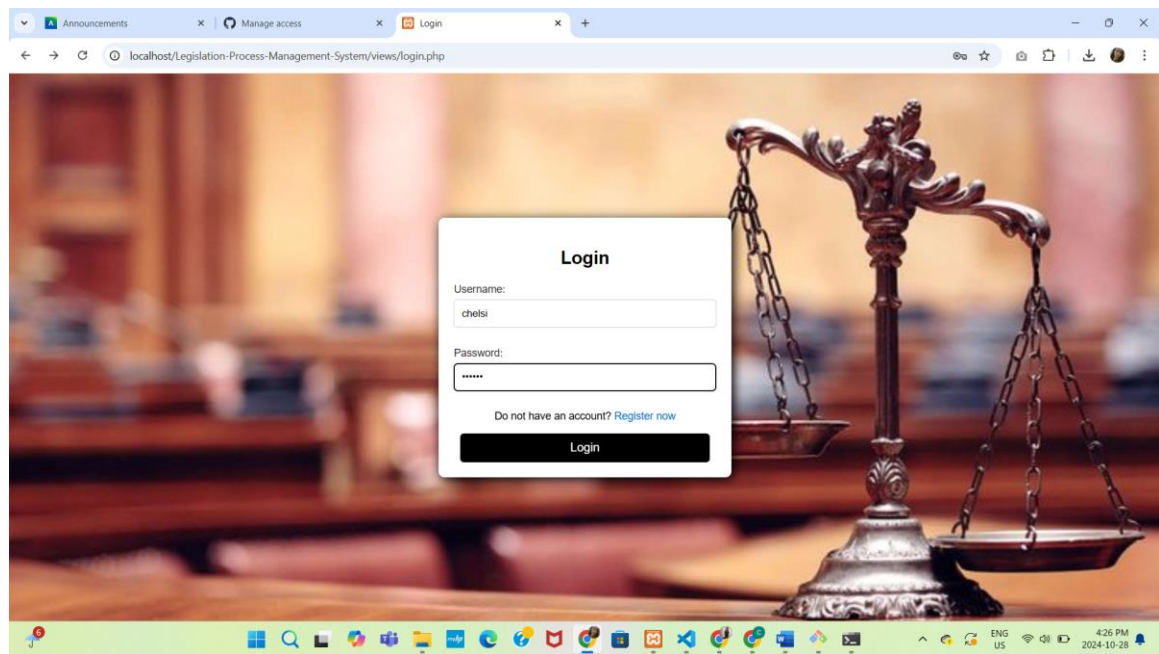
Registration Page



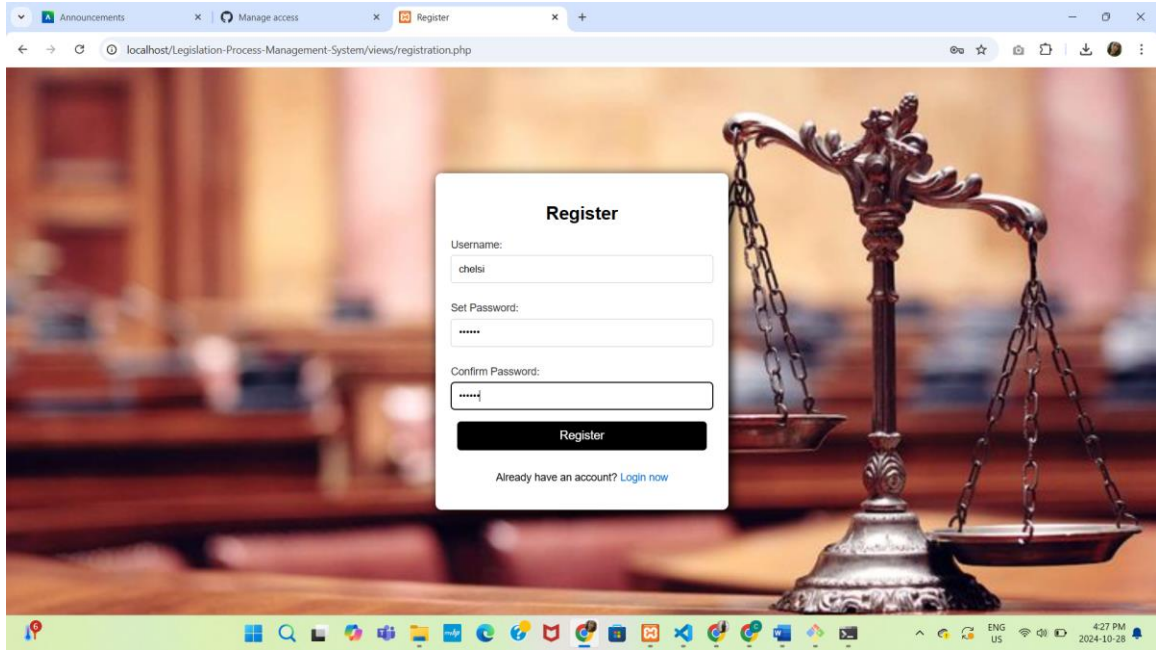
Incorrect Username/Password



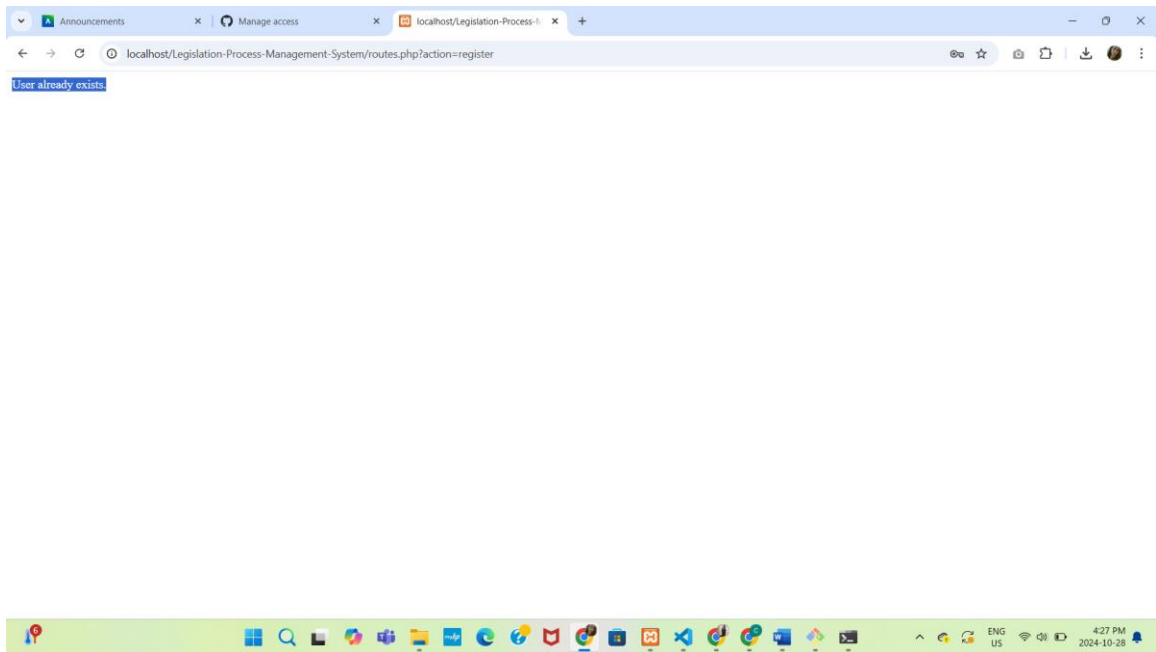
Successful Login



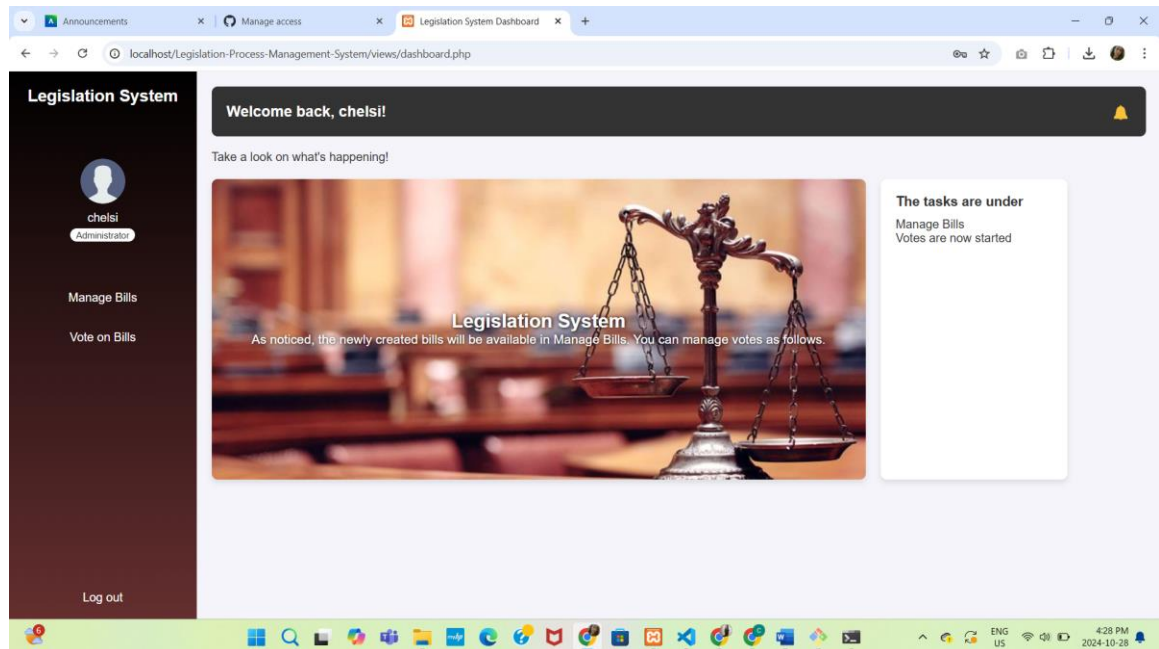
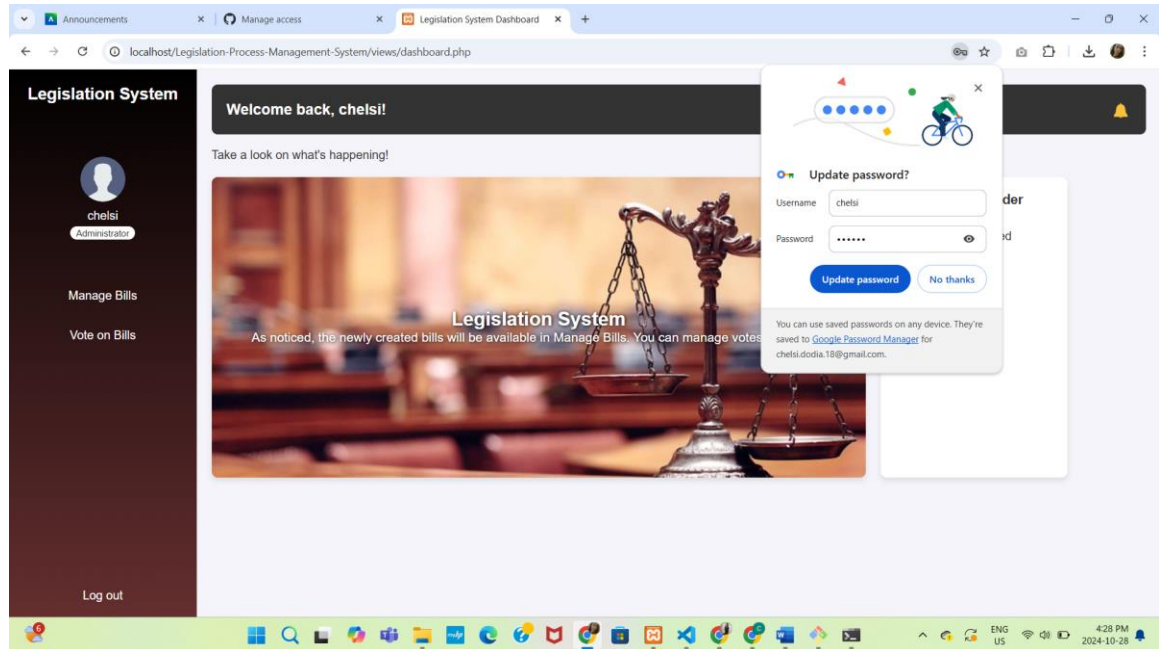
Successful Registration



User already exists

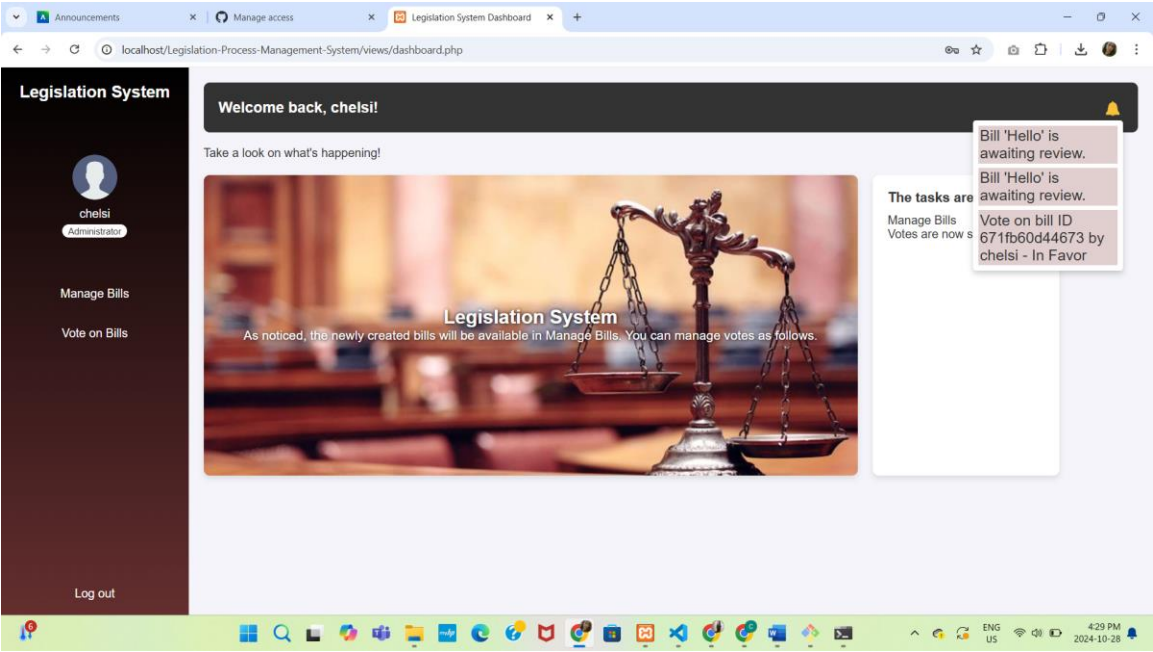


Dashboard (After successful Login)

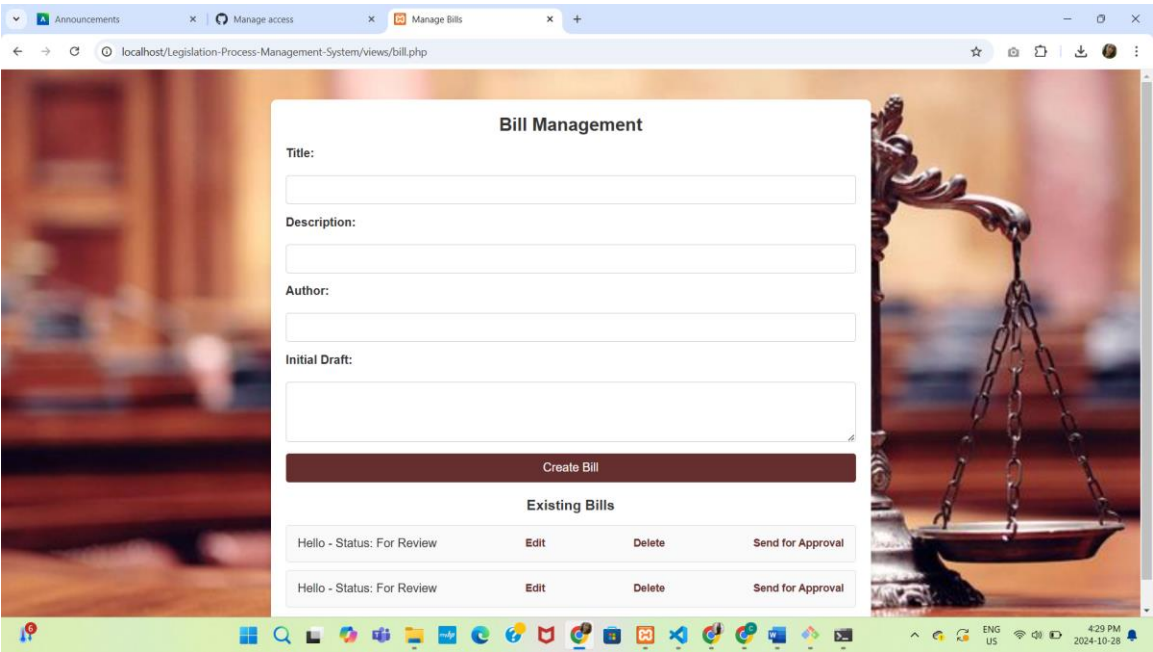


Here, there's a welcome back message along with user-type under the user image icon.

Notifications



Manage Bills



Add Bill

AnnouncementsManage accessManage Bills

localhost/Legislation-Process-Management-System/views/bill.php

Bill Management

Title:

Demo Bill

Description:

This is a demo Bill

Author:

Chelsi

Initial Draft:

Demo Bill Initial Draft

Create Bill

Existing Bills

Hello - Status: For Review

Edit

Delete

Send for Approval

Hello - Status: For Review

Edit

Delete

Send for Approval

4:30 PM

2024-10-28

AnnouncementsManage accessManage Bills

localhost/Legislation-Process-Management-System/views/bill.php

Bill Management

Title:

Description:

Author:

Initial Draft:

Create Bill

Existing Bills

Hello - Status: For Review

Edit

Delete

Send for Approval

Hello - Status: For Review

Edit

Delete

Send for Approval

Demo Bill - Status: Pending

Edit

Delete

Send for Approval

4:30 PM

2024-10-28

Demo Bill Creation Notification

Announcements

Manage access

Legislation System Dashboard

localhost/Legislation-Process-Management-System/views/dashboard.php

Legislation System

chelsi

Administrator

Manage Bills

Vote on Bills

Log out

Welcome back, chelsi!

Take a look on what's happening!

Legislation System

As noticed, the newly created bills will be available in Manage Bills. You can manage votes as follows.

The tasks are

Manage Bills

Votes are now s

Bill 'Hello' is awaiting review.

Bill 'Hello' is awaiting review.

Bill 'Demo Bill' is awaiting review.

Vote on bill ID 671fb60d44673 by chelsi - In Favor

Vote

Announcements

Manage access

Vote on Bills

localhost/Legislation-Process-Management-System/views/voting.php

Vote on Bills

Hello - Status: For Review

Vote Now

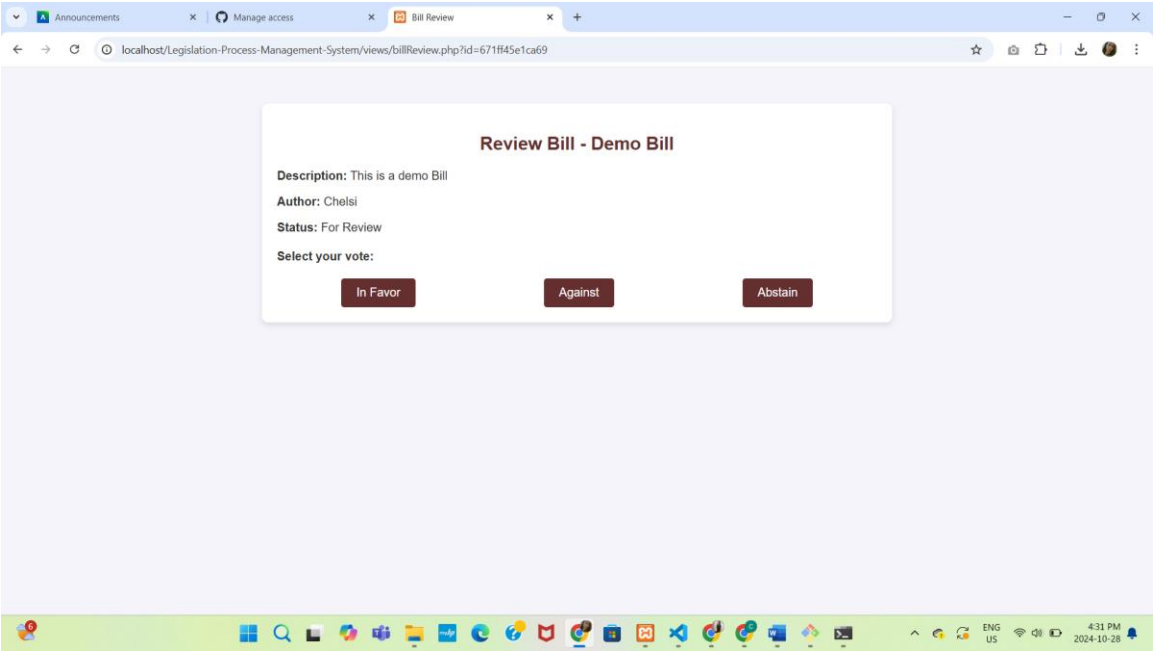
Hello - Status: For Review

Vote Now

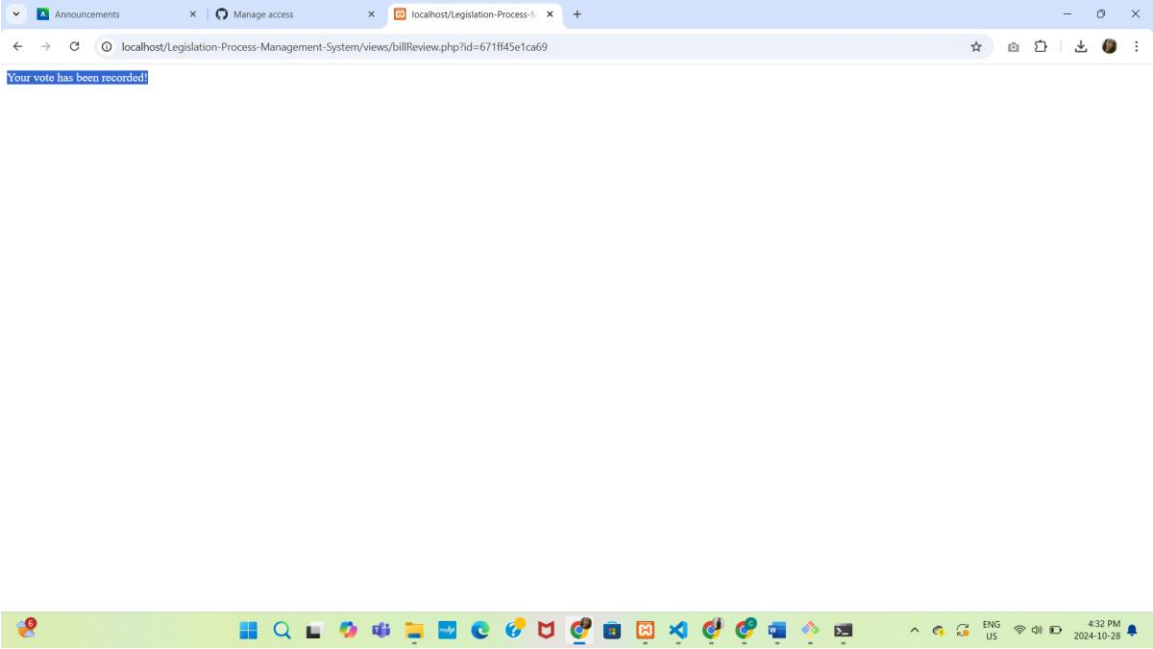
Demo Bill - Status: For Review

Vote Now

Vote Now



Voting as In Favour (Vote Recorder)



Voting Notified

Announcements

Manage access

Legislation System Dashboard

localhost/Legislation-Process-Management-System/Views/dashboard.php

Legislation System

chelsi

Administrator

Manage Bills

Vote on Bills

Log out

Welcome back, chelsi!

Take a look on what's happening!

Legislation System

As noticed, the newly created bills will be available in Manage Bills. You can manage votes as follows.

The tasks are

Manage Bills

Votes are now s

Bill 'Hello' is awaiting review.

Bill 'Hello' is awaiting review.

Bill 'Demo Bill' is awaiting review.

Vote on bill ID 671fb60d44673 by chelsi - In Favor

Vote on bill ID 671ff45e1ca69 by chelsi - In Favor

ENG US

4:33 PM

2024-10-28

Set Up and Run the System

Setup Steps

1. Clone the Repository:

```
bash
git clone https://github.com/your-repo/legislation-process-management.git
cd legislation-process-management
```

2. Install Dependencies:

```
bash
composer install
```

3. Run the Application:

```
Start the PHP built-in server:
bash
php -S localhost:8000 -t public
```

4. Access the Application:

- Open your browser and navigate to `http://localhost:8000`.